Programmierung 2 VU 051020

Exercise unit 2

WS 2026



universität
wien

Vector – Overview

Based on the Vector class as described in the slides for week 1, the following must be completed by next week

Iterators **Iterator** and **ConstIterator**. Enable the use of range-based-for-loops and STL algorithms.

Testing Test your implementation with the available test programs (see learning platform).

universität wien

Iterators – Repetition

- Iterators are a generalization of the pointer concept (or the reference concept). An iterator references a value in a container. At least the two operators * (dereference) and ++ (prefix increment; switching to the next value) are offered.
- Further operations are optional. In the context of range-based-for-loops, != (comparison of two iterators) is necessary (usually == is also offered).
- The C++ standard defines several Iterator types that offer different operations. It is precisely defined which operations a certain type must offer. The details are complex.
- Our iterators are forward iterators.

Compatibility of our iterators with STL algorithms

In order for iterators to be usable by STL algorithms, some type aliases must be created for the iterators.

The easiest way to define these is at the beginning of the vector class:

```
class Vector{
public:
  class ConstIterator;
  class Iterator;
  using value_type = double;
  using size_type = std::size_t;
  using difference_type = std::ptrdiff_t;
  using reference = value_type&;
  using const_reference = const value_type&;
  using pointer = value_type*;
  using const_pointer = const value_type*;
  using iterator = Vector::Iterator;
  using const_iterator = Vector::ConstIterator;
private:
  //Instance variables
public:
  //Methods
};
```

- Make sure that your Vector class only uses the data types from the using declarations.

Vorsicht: const_iterator $\neq$ const iterator

A const_iterator is not a constant iterator!

Instead, a const_iterator provides a constant reference to the referenced value. This means that the dereference operation (operator$*$) only allows read access to the element stored in the vector.

const_iterator is therefore an independent class with its own implementation.

universität wien

Iterator

```
class Vector {
  (...)
  class Iterator {
    public:
      using value_type = Vector::value_type;
      using reference = Vector::reference;
      using pointer = Vector::pointer;
      using difference_type = Vector::difference_type;
      using iterator_category = std::forward_iterator_tag;
    private:
    //Instance variables
    public:
    //Methods
  };
  class ConstIterator {
    public:
      using value_type = Vector::value_type;
      using reference = Vector::const_reference;
      using pointer = Vector::const_pointer;
      using difference_type = Vector::difference_type;
      using iterator_category = std::forward_iterator_tag;
    private:
    //Instance variables
    public:
    //Methods
  };
};
```

universität wien

Iterators for our Vector class

Extend the Vector class with the begin() and end() methods.

iterator begin() Returns an iterator on the first element in the vector. If the vector is empty, it corresponds to the end iterator

iterator end() Returns an iterator on the element after the last element in the vector.

const_iterator begin() const Returns an iterator on the first element in the vector. If the vector is empty, it corresponds to the end iterator

const_iterator end() const Returns an iterator on the element after the last element in the vector.

universität wien

Iterator – Implementation

Instance variables

pointer ptr	Pointer to an element in the vector.

Constructors

Default	Returns an iterator on **nullptr**.
pointer ptr	Returns an iterator that sets the instance variable to ptr.

Methods

reference operator*() const?

pointer operator->() const?

bool operator==(const const_iterator&) const?

bool operator!=(const const_iterator&) const?

iterator& operator++() const? (Prefix)

iterator operator++(int) const? (Postfix)

operator const_iterator() const? (Typ-Konversion)

Vector::difference_type operator-(const const_iterator&) const?

universität wien

ConstIterator – Implementation

Instance variables

pointer ptr	Zeiger auf ein Element im Vector.

Constructors

Default	Returns a ConstIterator on **nullptr**.
pointer ptr	Returns a ConstIterator that sets the instance variable to ptr.

Methods

reference operator*() const?

pointer operator->() const?

bool operator==(const const_iterator&) const?

bool operator!=(const const_iterator&) const?

const_iterator& operator++() const? (Prefix)

const_iterator operator++(int) const? (Postfix)

Vector::difference_type operator-(const const_iterator&) const?

universität wien

Iterators – Remarks

- Extend the Vector class with the methods **iterator begin()**, **iterator end()**, as well as the corresponding const methods for const_iterator.
- Think about which of the (const)iterator methods must be **const** and which must not.
- What is the const_iterator type conversion operator needed for in the iterator class?
- Why do the comparison operators == or != of the iterator class use **const_iterator** as parameter?

Vector – Insert/Erase

The methods `insert` and `erase` can be copied from the vector specification. For methods `insert` and `erase` to work, the method `operator-` must be defined in `Iterator` and `ConstIterator`.
Note: The versions of `insert` and `erase` on Moodle should be easier to copy.

universität wien

Vector – Test files

You should write your own main file to test your implementation. Test files are provided every week as a supplement. Please note that these are only intended as an aid and **do not guarantee** that your Vector will work correctly.

universität wien

Description Test files – Vector & Iterators (1)

You will find the following test files on Moodle. Try to compile and bind the files with your Vector class.

If **types_test.cpp** does not compile, you have not (properly) defined at least one of the type aliases required in our context.

If **const_test.cpp** or **const_test_arrow.cpp** compiles, you have not implemented const_iterator properly or (even worse) have not written your own `ConstIterator` class. Run the program and think about why the result should not actually be like this.

universität wien

Vector – Extensions

You can implement further methods and global functions for practice purposes (see lecture slides PR1). Please also note the exercises on the VO slides.
The following (mandatory) extension will be introduced in the next exercise session:

- Templates allow you to store "any" values, not just double values. (week 3)

For the test, you need a vector with the basic functionality described, iterators and templates.

universität wien